

- Description

This module manages an electric vehicle (EV) charging station, ensuring that:

1. **Vehicles are assigned to available charging slots based on queue order.**
2. **Each charging session is monitored**, ensuring the correct charging time and cost calculation.
3. **If all slots are occupied, new vehicles are added to a waiting queue.**
4. **Once charging is complete, the next vehicle in the queue is assigned a slot.**
5. **A log of all charging transactions** is maintained for record-keeping and billing.

This system ensures efficient charging slot allocation, fair queue management, and accurate billing.

- Specification

module EVChargingSystem

var chargingSlots: integer

var waitingQueue: sequence of (string, real)

var activeCharging: sequence of (string, real)

var transactionLog: sequence of string

process requestCharging(licensePlate: string, chargeTime: real) flag: boolean

functional scenario form:

|activeCharging| < chargingSlots && flag = true and activeCharging = activeCharging $\oplus$ (licensePlate, chargeTime), Charging slot assigned successfully.

||

|activeCharging| >= chargingSlots && flag = false and waitingQueue = waitingQueue $\oplus$ (licensePlate, chargeTime), All slots occupied! Vehicle added to waiting queue.

||

chargeTime <= 0 && flag = false, Invalid charging time! Request rejected.

end process

process completeCharging(licensePlate: string)

functional scenario form:

(licensePlate $\in$ activeCharging) && activeCharging = remove(activeCharging, licensePlate $\wedge$ transactionLog = transactionLog $\oplus$ "Charging completed for " + licensePlate $\wedge$ (if |waitingQueue| > 0 then activeCharging = activeCharging $\oplus$ first(waitingQueue)) Charging completed, slot freed, next in queue assigned.

OR

(licensePlate $\notin$ activeCharging) && Vehicle not found in active charging slots.

end process

end module

- Java Code

```
package code;
import java.util.*;
```

```

class Vehicle {
    private String licensePlate;
    private double chargeTime;

    public Vehicle(String licensePlate, double chargeTime) {
        this.licensePlate = licensePlate;
        this.chargeTime = chargeTime;
    }
    public String getLicensePlate() {
        return licensePlate;
    }
    public double getChargeTime() {
        return chargeTime;
    }
}

class ChargingStation {
    private int totalSlots;
    private double costPerHour;
    private Queue<Vehicle> waitingQueue;
    private Map<String, Double> activeCharging;
    private List<String> transactionLog;
    public ChargingStation(int totalSlots, double costPerHour) {
        this.totalSlots = totalSlots;
        this.costPerHour = costPerHour;
        this.waitingQueue = new LinkedList<>();
        this.activeCharging = new HashMap<>();
        this.transactionLog = new ArrayList<>();
    }
    public void requestCharging(String licensePlate, double chargeTime)
    {
        if (chargeTime <= 0) {
            System.out.println("Invalid charge time! Must be greater th
an zero.");
            return;
        }
        if (activeCharging.size() < totalSlots) {
            activeCharging.put(licensePlate, chargeTime);
            System.out.printf("Vehicle %s assigned to charging slot for
%.2f hours.%n", licensePlate, chargeTime);
        } else {
            waitingQueue.add(new Vehicle(licensePlate, chargeTime));
            System.out.printf("All slots occupied! Vehicle %s added to
waiting queue.%n", licensePlate);
        }
    }
}

```

```

    }
    public void completeCharging(String licensePlate) {
        if (activeCharging.containsKey(licensePlate)) {
            double chargeTime = activeCharging.remove(licensePlate);
            double cost = chargeTime * costPerHour;
            transactionLog.add(String.format("Charged: %s - %.2f hours,
Cost: $%.2f", licensePlate, chargeTime, cost));
            System.out.printf("Charging completed for %s. Cost: $%.2f%n",
licensePlate, cost);
            if (!waitingQueue.isEmpty()) {
                Vehicle nextVehicle = waitingQueue.poll();
                activeCharging.put(nextVehicle.getLicensePlate(), nextV
ehicle.getChargeTime());
                System.out.printf("Next in queue: %s assigned to chargi
ng slot.%n", nextVehicle.getLicensePlate());
            }
        } else {
            System.out.println("Vehicle not found in active charging sl
ots.");
        }
    }
    public void printTransactionLog() {
        System.out.println("\nCharging Transaction Log:");
        for (String log : transactionLog) {
            System.out.println(log);
        }
    }
}

public class c12_EVChargingSystem {
    public static void main(String[] args) {
        ChargingStation station = new ChargingStation(2, 5.0);
        // testcase-VT:
        // station.requestCharging("ABC123", 2.3);
        // station.requestCharging("XYZ789", 1.2);
        // station.requestCharging("LMN456", 5.5);
        // station.completeCharging("ABC123");
        // station.completeCharging("XYZ789");
        // station.completeCharging("LMN456");
        // station.completeCharging("LMN456");
        // station.requestCharging("QWE222", -1.9);

        // testcase-FT:
        // station.requestCharging("", 0);
        // station.requestCharging("z?d", 1.3951725578848605E298);
    }
}

```

```
//      station.requestCharging("", -1.7976931348623157E308);
//      station.completeCharging("");
//      station.completeCharging("z?d");
//      station.completeCharging("");
//      station.completeCharging("deuu");
//      station.requestCharging("deUU", 0);

//      // testcase-Z3:
//      station.requestCharging("ABC123", 4);
//      station.requestCharging("XYZ789", -1);
//      station.requestCharging("LMN456", 101);
//      station.requestCharging("ABC123", -1);
//      station.requestCharging("XYZ789", 101);
//      station.requestCharging("LMN456", 9);
//      station.requestCharging("QWE222", 11);
//      station.requestCharging("QWE222", 9);

//      // testcase-UVT:
station.requestCharging("ABC123", 0);
station.requestCharging("XYZ789", 1.3);
station.requestCharging("LMN456", -1.79);
station.completeCharging("ABC123");
station.completeCharging("XYZ789");
station.completeCharging("LMN456");
station.completeCharging("QWE222");
station.requestCharging("QWE222", 0);

station.printTransactionLog();
}
}
```

- Coverage report

1. Test case number: 8
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.2 %	10	824	834
▼ src	1.2 %	10	824	834
▼ code	1.2 %	10	824	834
▼ c12_EVChargingSystem.java	100.0 %	10	0	10
> c12_EVChargingSystem		0	0	0
> ChargingStation	100.0 %	10	0	10
> Vehicle		0	0	0

3. Vibration testing instruction coverage: 98.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.5 %	240	15,472	15,712
▼ src	1.5 %	240	15,472	15,712
▼ code	1.5 %	240	15,472	15,712
▼ c12_EVChargingSystem.java	98.8 %	240	3	243
> c12_EVChargingSystem	92.5 %	37	3	40
> ChargingStation	100.0 %	188	0	188
> Vehicle	100.0 %	15	0	15

4. Fuzz testing branch coverage: 80%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	826	834
▼ src	1.0 %	8	826	834
▼ code	1.0 %	8	826	834
▼ c12_EVChargingSystem.java	80.0 %	8	2	10
> ChargingStation	80.0 %	8	2	10
> c12_EVChargingSystem		0	0	0

5. Fuzz testing instruction coverage: 74.1%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.1 %	180	15,532	15,712
▼ src	1.1 %	180	15,532	15,712
▼ code	1.1 %	180	15,532	15,712
▼ c12_EVChargingSystem.java	74.1 %	180	63	243
> ChargingStation	76.1 %	143	45	188
> c12_EVChargingSystem	92.5 %	37	3	40

6. Z3 branch coverage: 50%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.6 %	5	835	840
▼ src	0.6 %	5	835	840
▼ code	0.6 %	5	835	840
▼ c12_EVChargingSystem.java	50.0 %	5	5	10
> ChargingStation	50.0 %	5	5	10
> c12_EVChargingSystem		0	0	0
> Vehicle		0	0	0

7. Z3 instruction coverage: 57.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.6 %	143	21,954	22,097
▼ src	0.6 %	143	21,954	22,097
▼ code	0.6 %	143	21,954	22,097
▼ c12_EVChargingSystem.java	57.9 %	143	104	247
> ChargingStation	49.5 %	93	95	188
> Vehicle	60.0 %	9	6	15
> c12_EVChargingSystem	93.2 %	41	3	44

8. Unimproved V-method branch coverage: 80%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	832	840
▼ src	1.0 %	8	832	840
▼ code	1.0 %	8	832	840
▼ c12_EVChargingSystem.java	80.0 %	8	2	10
> ChargingStation	80.0 %	8	2	10
> c12_EVChargingSystem		0	0	0

9. Unimproved V-method instruction coverage: 74.1%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.8 %	180	21,913	22,093
▼ src	0.8 %	180	21,913	22,093
▼ code	0.8 %	180	21,913	22,093
▼ c12_EVChargingSystem.java	74.1 %	180	63	243
> ChargingStation	76.1 %	143	45	188
> c12_EVChargingSystem	92.5 %	37	3	40